WG.48.3: Independence of Textual Order of Paragraphs

Proposal for the Z Standard, by Wolfgang Grieskamp (wg@cs.tu-berlin.de), November 11, 1998, (v3)

Note This is a technical refinement of an earlier proposal of SHV.

What

The order how paragraphs appear in a Z section is arbitrary. The definition-use relation of paragraphs must be acyclic. (A paragraph is in definition-use relation to another paragraph, if it introduces a name which is referred to by the other paragraph.)

Why

Shortly: the adequate way to structure the paragraphs of a section is not always bottom-up. (As far as I know, the panel already agreed that paragraph reordering is useful in general.)

How

Conformance Tools working in batch mode which don't implement this feature are considered still as conforming, since filters can be build which perform the paragraph reordering for them. (However, these filters need to type-check the specification; see discussion of implementation below.)

Dependencies Proposal **WG.48.2** is assumed, such that redefinition of Δ and Ξ is prohibited.

Updates to WD 1.5

- Lexis, Clause 7.4.4 (User-defined operators): Refine:

  > [...] These associations extent over the entire Z section they are defined within and are disabled at the end of the section. [...]

- Type Inference Rules, Clause 13.2.2.1 (the according clause in 12, resp.): add a comment such as:

  > The order how paragraphs appear in a Z section is arbitrary. The definition-use relation of paragraphs must be acyclic. (A paragraph is in definition-use relation to another paragraph, if it introduces a name which is referred to by the other paragraph.)

  Define reordering (and cycle check) as a condition to the rule. This could be something along:

$$\frac{\begin{array}{c}\Gamma_0 \bullet^{\mathbf{D}} d_{\rho(1)} \\ \dots \\ \Gamma_{\rho(\mathbf{n}-1)} \bullet^{\mathbf{D}} d_{\rho(\mathbf{n})}\end{array}}{\dots} \qquad \begin{array}{l}\dots \\ \rho : 1 \mathinner{..} n \mathrel{\text{Œ}} 1 \mathinner{..} n\end{array}$$

  It's sufficient to just demand that the re-ordering ρ is a bijection, since:

– any instance of ρ which does not represent a proper bottom-up order is rejected by clause 13.2.5.1, which would detect that a referenced name i is not defined. It doesn't matter for the rule system that this undefinedness results from that i is never defined, that i is defined later on in the order, or that i is cyclic defined.

– if there are degrees of freedom how to reorder paragraphs, then which order to choose doesn't matters semantically, as justified below.

Semantics Consider there are two possible well-formed (well-typed) re-orderings S_1 and S_2 of a section. Are these always semantically equivalent? Here is a justification which actually reduces the problem to a desirable property of the semantic equations in general (which seems to be fulfilled in WD 1.5 – yet this still needs to be counter-checked by the authors of clause 16).

From any section, one can syntactically build what I would call the *algebraic normal form*, $ANF_{\mathbf{S}} = (G_{\mathbf{S}}, C_{\mathbf{S}}, P_{\mathbf{S}})$, where $G_{\mathbf{S}}$, $C_{\mathbf{S}}$, and $P_{\mathbf{S}}$ are sets of paragraphs, which are constructed as follows:

- $G_{\mathbf{S}}$ contains a given type paragraph for each given type and free type paragraph in S;
- $C_{\mathbf{S}}$ contains for each axiomatic definition in S a paragraph with the declarations of its global constants with their basic types (build from given types, formal generics, powerset, and schema type);
- $P_{\mathbf{S}}$ contains for each axiomatic definition in S a paragraph with the literal properties of it, enriched by properties induced by the declarations (e.g. for $x : \mathbb{I}$ the property $x \in \mathbb{I}$). For free types, $P_{\mathbf{S}}$ contains the set of according free-type axioms.

From $ANF_{\mathbf{S}}$ one can reconstruct a Z sections $S' = \mathsf{section}\ i\ \mathsf{parents}\ i_1, \ldots, i_{\mathbf{n}}\ \mathsf{END}\ G_{\mathbf{S}}\ C_{\mathbf{S}}\ P_{\mathbf{S}}\ \mathsf{END}$, where the individual paragraphs from $G_{\mathbf{S}}$ (resp. $C_{\mathbf{S}}$ and $P_{\mathbf{S}}$) are ordered arbitrarily, and which is by construction definition-before-use. Now we *assume that the semantics equations ensure* that for each arbitrary arrangement of S' (according to the degrees of freedom in ordering individual paragraphs from $G_{\mathbf{S}}$, $C_{\mathbf{S}}$, and $P_{\mathbf{S}}$) it holds: $⦇S⦈^{\mathcal{S}}_{\mathcal{C}} = ⦇S'⦈^{\mathcal{S}}_{\mathcal{C}}$ (where S is well-typed). So we can lift the semantic mapping to ANFs, and say that $⦇S⦈^{\mathcal{S}}_{\mathcal{C}} = ⦇ANF_{\mathbf{S}}⦈^{\mathcal{S}}_{\mathcal{C}}$. Now since $ANF_{\mathbf{S}_1} = ANF_{\mathbf{S}_2}$ if S_1 and S_2 are reorderings of the same section, which is obvious since for the construction of the ANF the order how paragraphs are added to the sets $G_{\mathbf{S}}$ (resp. $C_{\mathbf{S}}$ and $P_{\mathbf{S}}$) doesn't matters, we can conclude that $⦇S_1⦈^{\mathcal{S}}_{\mathcal{C}} = ⦇ANF_{\mathbf{S}_1}⦈^{\mathcal{S}}_{\mathcal{C}} = ⦇ANF_{\mathbf{S}_2}⦈^{\mathcal{S}}_{\mathcal{C}} = ⦇S_2⦈^{\mathcal{S}}_{\mathcal{C}}$.

Implementation (Operator Templates) Since the token mapping introduced by operator templates needs to hold over an entire section, tools need to evaluate the templates before actual parsing begins. This can be achieved by evaluating the syntax of operator templates already at scanning time.

If a ‡]⌈-markup-box would be introduced for the LaTeX-markup in Annex A.3, then inside of this markup template paragraphs should not allowed to be mixed with other paragraphs. A lexer would not be able to recognise the operator templates easily, since hard-newlines separating paragraphs are converted to soft-newlines in dependency of the keywords introduced by templates.

Implementation (Type Inference) As can be drawn from the literature about type-inference for functional languages, it is not possible to check the unordered paragraphs of a section "at once" by global type unification. Trying so what amount to type inference in the presence of polymorphic recursion, which is not decidable. (The problem has been merely depicted in WD 1.5, Sect. 13.3, p67: "[need to] finalise the type of a generic definition before type-checking instantiations of that generic definition".)

As well, it is not possible to reorder paragraphs prior to type-checking, since the construction of the definition-use relation of paragraphs requires type information. This is a consequence of the fact that in Standard Z axiomatic definitions may include an arbitrary expression denoting a

schema in their declaration part. To derive what names are introduced, the type of the included expression needs to be inferred.

Therefore an implementation of paragraph reordering needs to be interleaved with type-checking. However, there is a simple model how this interleaving can be realized using a modular extension to existing type-checking algorithms: check each paragraph of a section by a *parallel executing thread*, and let these threads synchronise via the global constant environment Γ:

- Whenever a thread refers to a name i which is not yet define in Γ, it suspends waiting for a definition of i.
- Whenever a thread defines a name i in Γ, all threads waiting for this name are resumed.
- If all threads happen to wait for a name definition, a deadlock has occurred, which either reflects a cyclic definition are an undeclared-name error.

In practice, the handling of deadlocks has to be instrumented to produce meaningful error diagnostics.